14.7 Transactional Data Architecture

Overview

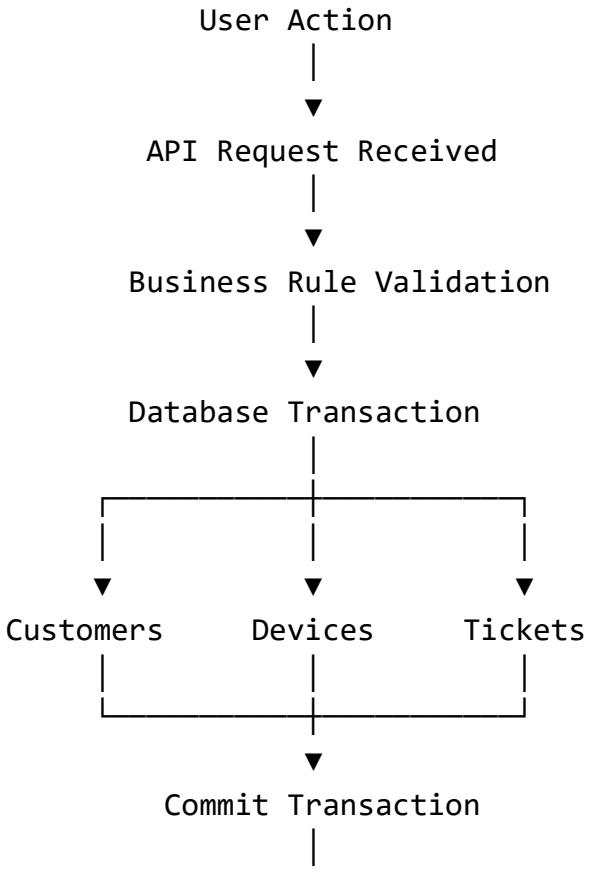
The Transactional Data Architecture defines how operational business data is created, updated, and maintained during the execution of platform services.

The architecture ensures that every business transaction is:

- Atomic
- Consistent
- Isolated
- Durable (ACID)

This guarantees that customer operations remain reliable even under high concurrency.

Transaction Flow



Transaction Boundaries

A business transaction may involve multiple entities but must behave as a single logical unit.

Examples:

Customer Registration

Creates:

- Customer
- Customer Profile
- Communication Preferences
- Initial Conversation Context

If any operation fails, the entire transaction is rolled back.

Ticket Creation

Creates:

- Ticket
- Initial Status
- Conversation Record
- AI Analysis
- Audit Event

The ticket is considered valid only when all associated records are successfully committed.

ACID Compliance

The transactional layer follows the ACID model:

Atomicity

All operations succeed or none are persisted.

Consistency

Every transaction preserves database integrity.

Isolation

Concurrent transactions do not interfere with each other.

Durability

Committed transactions survive system failures.

Transaction Types

The platform distinguishes between:

`Read Transaction`

`Write Transaction`

`Batch Transaction`

`AI-Assisted Transaction`

`Administrative Transaction`

`Background Transaction`

Each type follows its own optimization and validation strategy.

Concurrency Control

To maintain consistency under concurrent access, the platform supports:

- Optimistic locking where appropriate.
- Database constraints for integrity.
- Transaction isolation levels provided by PostgreSQL.
- Idempotent operations for retryable requests.

Event Generation

Successful transactions may emit domain events, such as:

`CustomerCreated`

`DeviceRegistered`

`TicketOpened`

`TicketClosed`

`KnowledgeUpdated`

`ConversationStarted`

These events can later be consumed by analytics, notifications, integrations, or future event-driven services without coupling them to the transactional workflow.

14.8 Analytical Data Architecture

Overview

While the Operational Data Store (ODS) supports day-to-day business operations, the Analytical Data Architecture is responsible for transforming operational data into strategic information.

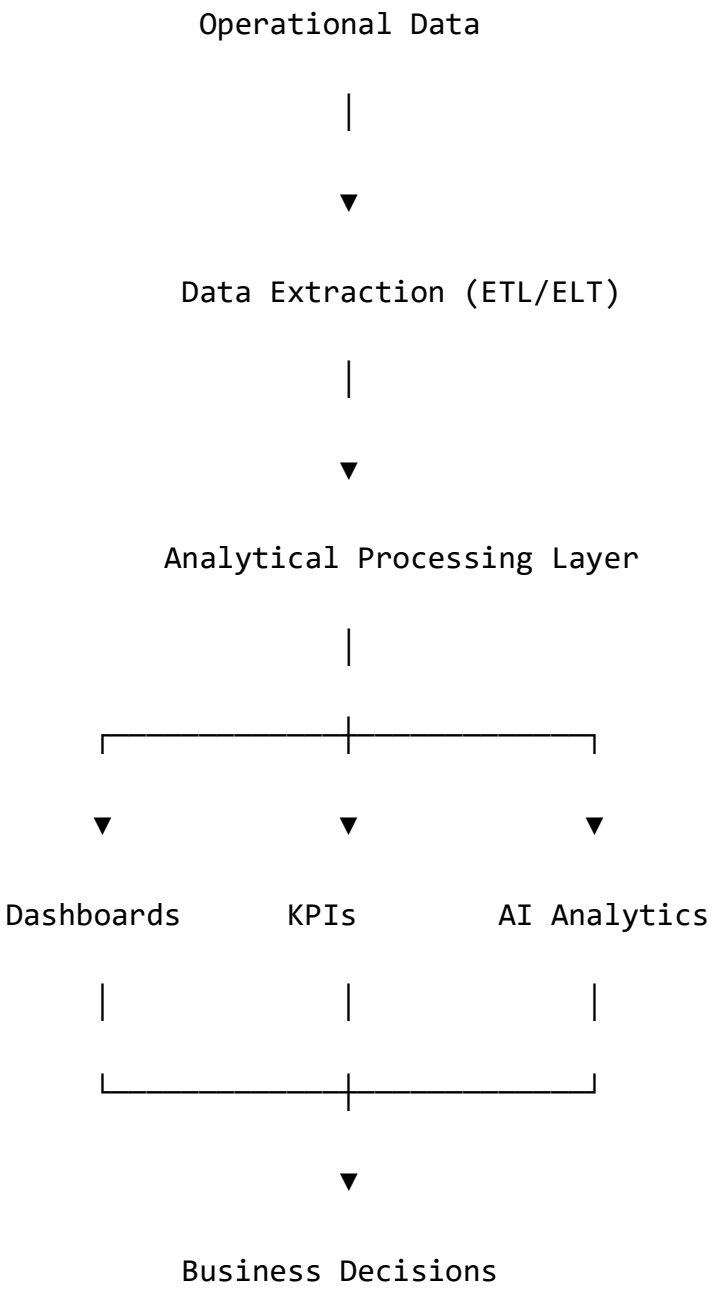
Its primary purpose is to support:

- Business Intelligence (BI)
- Executive dashboards
- Operational reporting
- Predictive analytics
- AI optimization

- Business decision-making

The analytical layer is optimized for reading, aggregation, and historical analysis rather than transactional processing.

Analytical Data Flow



Data Sources

Analytical data is derived from multiple operational domains.

Primary sources include:

```
Customers

Companies

Devices

Services

Tickets

Conversations

Knowledge Base

AI Events

Audit Logs

Notifications
```

These datasets are periodically consolidated into analytical models.

Analytical Models

The platform organizes analytical information into subject-oriented models.

Customer Analytics

Provides visibility into:

- Customer growth
- Active customers
- Retention rate
- Customer lifetime

- Customer satisfaction
- Communication channels

Service Analytics

Measures:

- Most requested services
- Average repair duration
- Service demand trends
- Revenue by service
- Technician workload

Ticket Analytics

Tracks:

- Open tickets
- Closed tickets
- Average resolution time
- SLA compliance
- Reopened tickets
- Escalation rate

AI Analytics

Monitors Bitey performance.

Metrics include:

- Intent detection accuracy
- Confidence score distribution
- Automatic ticket creation rate
- Knowledge retrieval effectiveness
- AI response latency
- AI-assisted resolution rate

Operational Analytics

Evaluates platform health.

Examples:

- API requests
- Error rate
- Authentication failures
- Active sessions
- Database performance
- Infrastructure utilization

Key Performance Indicators (KPIs)

The platform defines enterprise KPIs across all business domains.

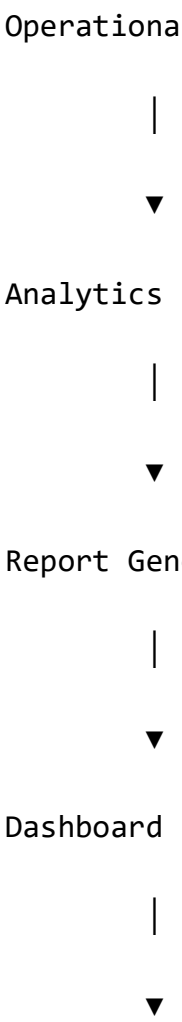
Examples:

```
Customer Satisfaction

Average Resolution Time

First Response Time

Ticket Resolution Rate

AI Automation Rate

Knowledge Usage Rate

Monthly Active Customers

Service Revenue

Platform Availability

Average API Response Time
```

KPIs are calculated from authoritative operational data to ensure consistency.

Reporting Architecture

Reports are generated through dedicated analytical services rather than directly querying transactional tables.



This separation minimizes the impact of reporting workloads on operational performance.

Historical Data

Analytical systems preserve historical snapshots to enable trend analysis.

Examples include:

- Ticket volume by month
- Service demand by quarter
- Customer growth over time
- AI performance evolution
- Revenue trends
- Device repair history

Historical records support forecasting and long-term strategic planning.

Future Business Intelligence

The architecture is designed to integrate with enterprise BI platforms such as:

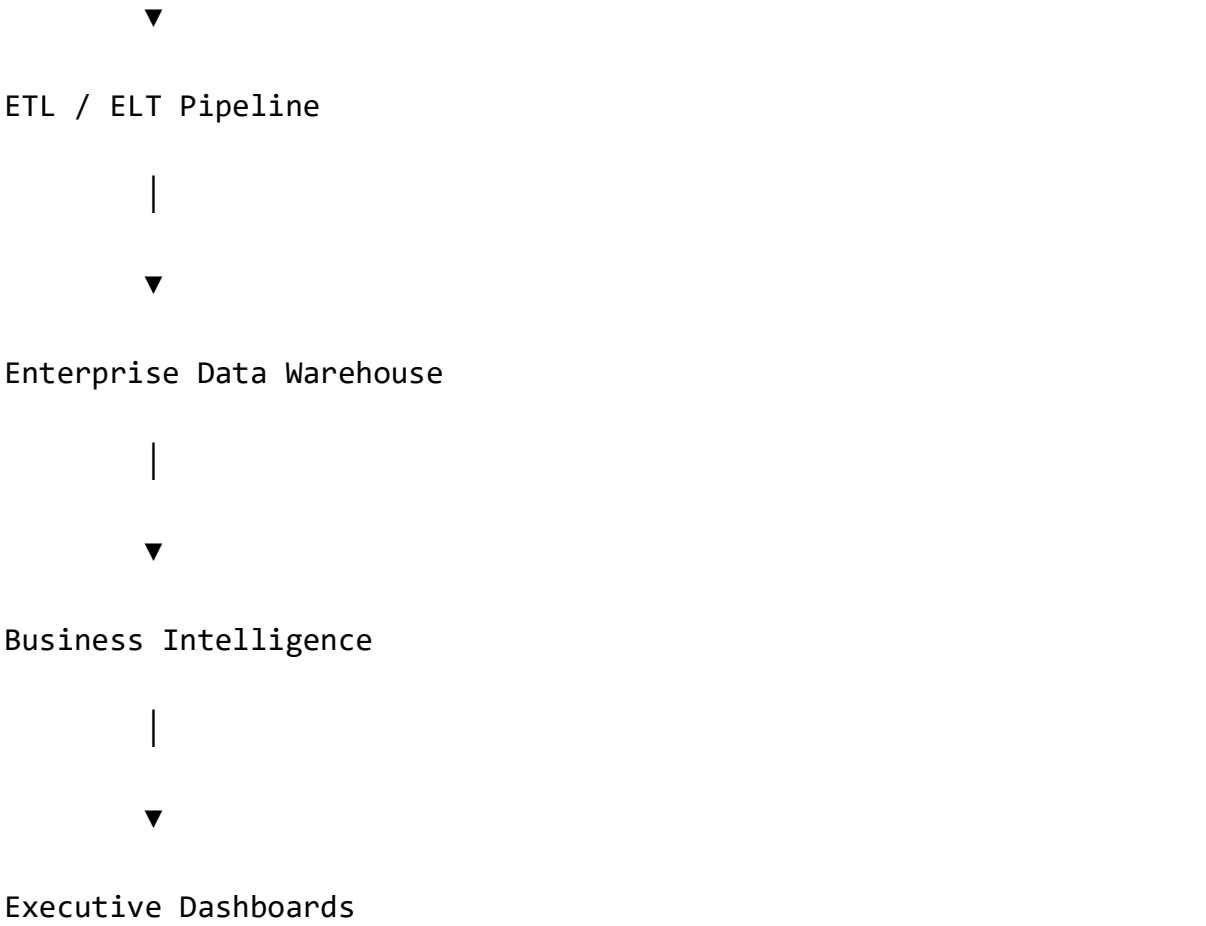
- Microsoft Power BI
- Grafana
- Metabase
- Apache Superset
- Future custom dashboards

This enables organizations to create interactive visualizations without modifying the operational backend.

Data Warehouse Readiness

As the platform grows, analytical workloads can be migrated to a dedicated data warehouse.

```
Operational Database
        |
```



This architecture supports scalability while preserving the performance of transactional operations.

Chapter Summary

The Analytical Data Architecture transforms operational data into actionable business intelligence. By separating transactional processing from analytical workloads, BiteFixes provides accurate reporting, enterprise KPIs, AI performance insights, and executive decision support while maintaining the responsiveness of the operational platform.

14.9 AI Memory Architecture

Overview

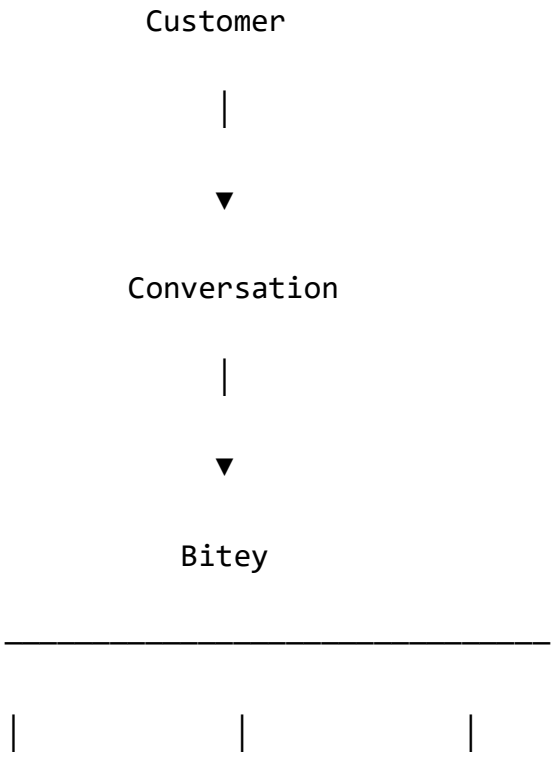
The AI Memory Architecture defines how Bitey acquires, stores, organizes, retrieves, updates, and utilizes knowledge during every interaction with customers, technicians, and companies.

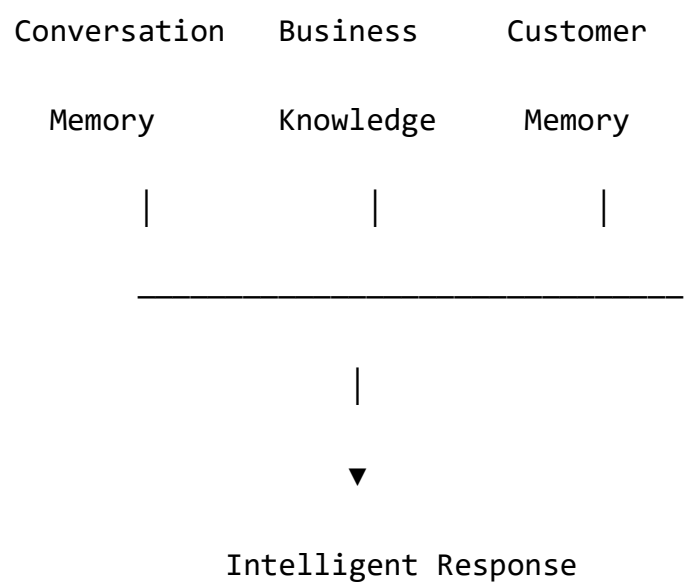
Unlike traditional applications, BiteFixes does not treat AI as a stateless request processor. Instead, Bitey maintains persistent contextual memory that evolves over time, allowing the platform to provide personalized, context-aware, and business-consistent assistance.

The architecture separates operational memory, conversational context, business knowledge, and long-term organizational memory to ensure scalability and maintainability.

AI Memory Vision

Bitey acts as the cognitive layer of the platform.

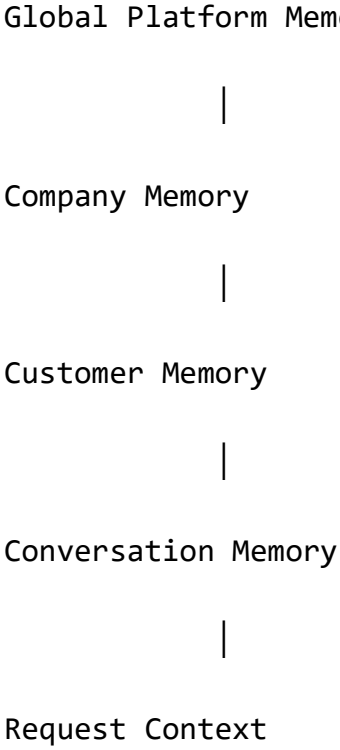




Every response is generated using multiple memory sources rather than relying solely on the current message.

Memory Hierarchy

The platform organizes memory into multiple layers.



Each layer provides additional context while preserving tenant isolation and data ownership.

Global Platform Memory

Global Memory contains platform-wide information that is shared across all tenants.

Examples include:

- Supported device categories
- Diagnostic methodologies
- Platform capabilities
- Generic troubleshooting procedures
- AI prompt templates
- System vocabulary
- Common technical concepts

Global Memory is maintained by the platform administrators and is read-only for tenant operations.

Company Memory

Each company maintains its own independent memory space.

Company Memory includes:

- Company profile
- Business policies
- Working hours
- Pricing rules
- Service catalog
- Internal procedures
- Brand identity
- Knowledge articles
- Frequently asked questions

No company can access another company's memory.

Customer Memory

Customer Memory stores long-term information associated with an individual customer.

Typical data includes:

- Contact information
- Preferred language
- Communication preferences
- Device ownership
- Repair history
- Previous conversations
- Satisfaction history
- Service preferences

This memory enables personalized interactions without requiring customers to repeat information across sessions.

Conversation Memory

Conversation Memory captures the context of an active interaction.

It includes:

- Current topic
- Recent messages
- Active ticket
- Detected intent
- Temporary variables
- Pending actions

```
Conversation
```



```
Recent Messages

Current Intent

Current Ticket

Pending Questions

Temporary Context
```

Conversation Memory is ephemeral and is refreshed as the dialogue progresses.

Operational Context

Every request generates a temporary execution context.

```
Authentication Context

Company Context
```

```
Customer Context

Conversation Context

Current Request

Business Rules

Security Context
```

This runtime context exists only during request processing and is discarded once the operation completes.

Knowledge Memory

Knowledge Memory stores structured business knowledge used for AI-assisted decision making.

Examples include:

- Repair procedures
- Troubleshooting guides
- Technical documentation
- Frequently asked questions
- Service descriptions
- Internal manuals

Knowledge is version-controlled and can be updated independently of the application code.

Episodic Memory

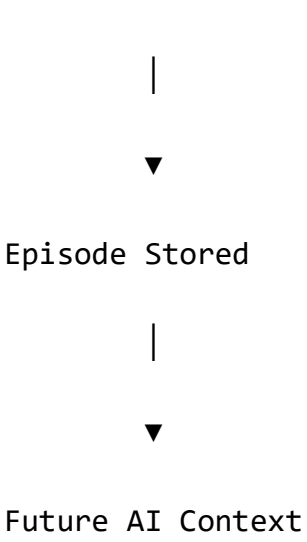
Episodic Memory records significant events experienced by the platform.

Examples:

- Ticket opened

- Ticket resolved
- Device repaired
- Customer complaint
- Successful recommendation
- Failed diagnosis

These events provide historical context for future interactions.



Semantic Memory

Semantic Memory represents generalized knowledge learned from accumulated information rather than individual events.

Examples include:

- Common failure patterns
- Frequently requested repairs
- Typical troubleshooting sequences
- Device reliability trends
- Common customer questions

Semantic Memory enables Bitey to provide increasingly accurate recommendations as the knowledge base evolves.

Memory Isolation

Memory isolation is enforced at every level.

```
Platform
    |
Company
    |
Customer
    |
Conversation
    |
Request
```

Isolation guarantees:

- Tenant separation
- Privacy protection
- Security compliance
- Context accuracy

No memory crosses tenant boundaries unless explicitly configured by platform administrators for shared global knowledge.

Chapter Summary

The AI Memory Architecture establishes Bitey as a stateful enterprise AI engine rather than a simple conversational interface. By organizing memory into hierarchical layers—including global, company, customer, conversation, operational, knowledge, episodic, and semantic memory—the platform delivers context-aware interactions while preserving security, scalability, and multi-tenant isolation. This architecture provides

the foundation for future capabilities such as semantic search, vector embeddings, intelligent recommendations, and autonomous AI agents.